



传送变长的信息（实现）

北京理工大学计算机学院
金旭亮

实现一问一答式的消息交换方式



一种简便的Client与Server数据通讯方式，就是一问一答，每次交换一个字符串。



两台计算机之间交换的信息，可以用“\r\n”（或“\n”）分隔。



使用BufferedReader.readLine()方法，可以很方便地从流中读一行（它会自动检测换行符，无需手工实现）。



使用BufferedWriter.write()方法，可以很方便地向流中写入一个字符串，之后，再调用它的新line()方法添加换行符。最后，调用flush()方法刷新数据缓存区。

示例分析：Server端

```
//基于换行符实现的“一对一通讯”
1 usage
public class EchoServer {
    3 usages
    public static final int SERVER_PORT = 10001;

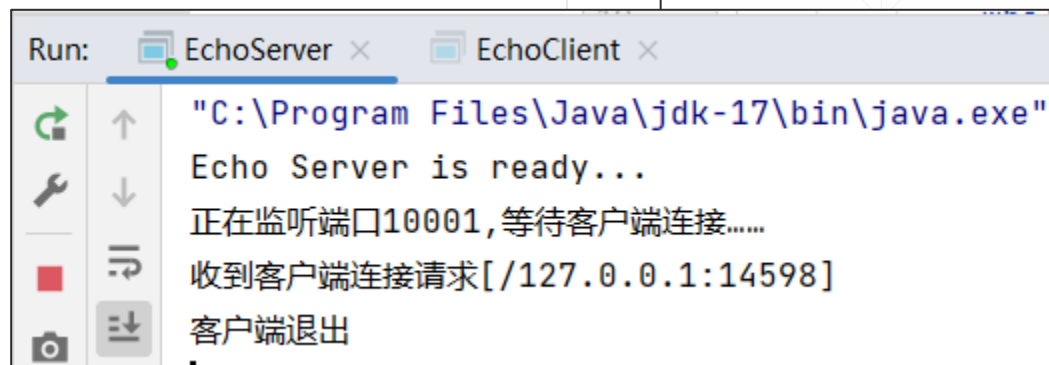
    public static void main(String[] args) throws IOException {
        System.out.println("Echo Server is ready...");
        try (var server = new ServerSocket(SERVER_PORT)) {
            System.out.println("正在监听端口" + SERVER_PORT + ",等待客户端连接.....");
            while (true) {
                try (var client :Socket = server.accept()) {
                    System.out.println("收到客户端连接请求[" + client.getInetAddress() + ":" + client.getPort() + "]");
                    var reader = new BufferedReader(new InputStreamReader(client.getInputStream(), StandardCharsets.UTF_8));
                    var writer = new BufferedWriter(new OutputStreamWriter(client.getOutputStream(), StandardCharsets.UTF_8));
                    //输出欢迎信息
                    writer.write( str: "欢迎访问鸚鵡学舌服务器, 请输入字符串, 输入"quit"退出。");
                    writer.newLine();
                    writer.flush();
                    //响应客户端请求
                    while (true){...}
                }
            }
        }
    }
}
```

不管是接收还是发送，都必须在消息末尾加上“\n”。

见下页

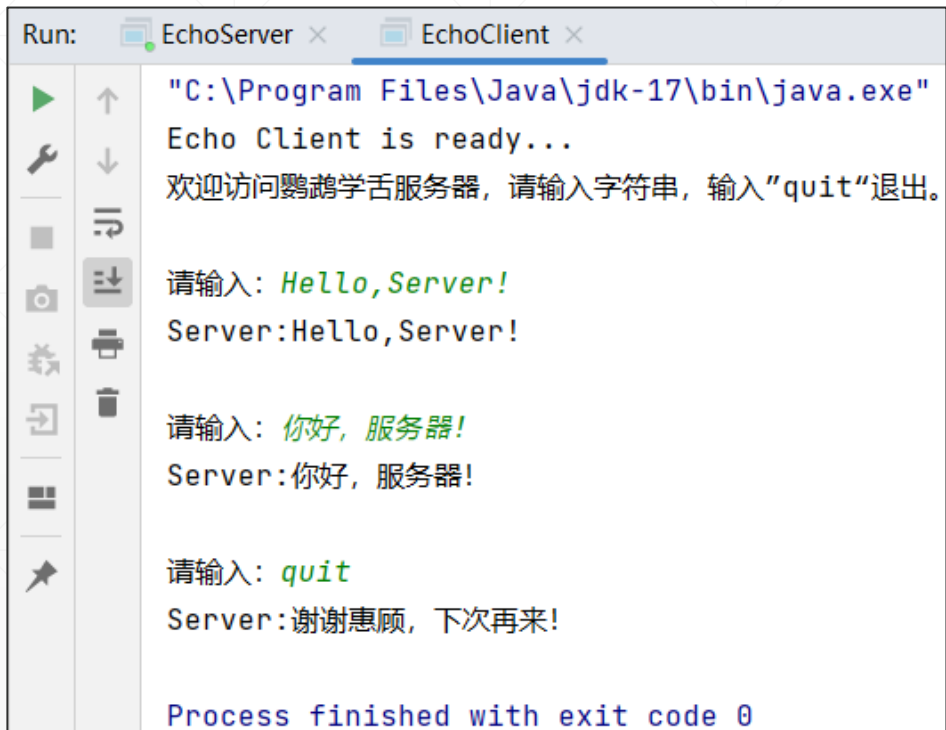
示例分析：Server端

```
System.out.println("正在监听端口" + SERVER_PORT + ",等待客户端连接.....");
while (true) {
    try (var client : Socket = server.accept()) {
        System.out.println("收到客户端连接请求[" + client.getInetAddress() + ":" + client.getPort() + "]");
        var reader = new BufferedReader(new InputStreamReader(client.getInputStream(), StandardCharsets.UTF_8));
        var writer = new BufferedWriter(new OutputStreamWriter(client.getOutputStream(), StandardCharsets.UTF_8));
        //输出欢迎信息
        writer.write(str: "欢迎访问鸚鵡学舌服务器, 请输入字符串, 输入\"quit\"退出。");
        writer.newLine();
        writer.flush();
        //响应客户端请求
        while (true){
            String message = reader.readLine();
            //结束服务, 断开客户端连接
            if(message.equalsIgnoreCase( anotherString: "quit")){
                System.out.println("客户端退出");
                writer.write(str: "Server:谢谢惠顾, 下次再来! ");
                writer.newLine();
                writer.flush();
                break;
            }
            //向客户端返回信息
            writer.write(str: "Server:" + message);
            writer.newLine();
            writer.flush();
        }
    }
}
```



```
Run: EchoServer x EchoClient x
"C:\Program Files\Java\jdk-17\bin\java.exe"
Echo Server is ready...
正在监听端口10001,等待客户端连接.....
收到客户端连接请求[/127.0.0.1:14598]
客户端退出
```

示例分析：Client端



```
Run: EchoServer x EchoClient x
"C:\Program Files\Java\jdk-17\bin\java.exe"
Echo Client is ready...
欢迎访问鹦鹉学舌服务器, 请输入字符串, 输入"quit"退出。

请输入: Hello, Server!
Server: Hello, Server!

请输入: 你好, 服务器!
Server: 你好, 服务器!

请输入: quit
Server: 谢谢惠顾, 下次再来!

Process finished with exit code 0
```

```
public class EchoClient {
    1 usage
    private static final String SERVER = "localhost";
    public static void main(String[] args) throws IOException {
        System.out.println("Echo Client is ready...");
        try (var socket = new Socket(SERVER, EchoServer.SERVER_PORT)) {
            var reader = new BufferedReader(new InputStreamReader(socket.getInputStream(),
                StandardCharsets.UTF_8));
            var writer = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream(),
                StandardCharsets.UTF_8));
            //接收服务端传回的信息
            String echo = reader.readLine();
            System.out.println(echo);
            while (true) {
                System.out.print("\n请输入: ");
                var consoleReader = new BufferedReader(new InputStreamReader(System.in));
                var userInput :String = consoleReader.readLine();
                //将用户从键盘输入的内容发送到Server端
                writer.write(userInput);
                writer.newLine();
                writer.flush();
                //接收服务端传回的信息
                echo = reader.readLine();
                System.out.println(echo);
                if (userInput.equalsIgnoreCase( anotherString: "quit")) {
                    //断开连接并退出
                    break;
                }
            }
        }
    }
}
```

使用transferTo()方法简化代码

```
public class SimplestEchoServer {  
    1 usage  
    public static final int SERVER_PORT = 10001;  
  
    public static void main(String[] args) throws IOException {  
        System.out.println("Echo Server is ready...");  
        try (var server = new ServerSocket(SERVER_PORT)) {  
            Socket client = server.accept();  
            InputStream in = client.getInputStream();  
            OutputStream out = client.getOutputStream();  
            var writer = new BufferedWriter(new OutputStreamWriter(out, StandardCharsets.UTF_8));  
            //输出欢迎信息  
            writer.write(str: "欢迎访问鹦鹉学舌服务器, 请输入字符串, 输入\"quit\"退出。");  
            writer.newLine();  
            writer.flush();  
            while (true) {  
                //使用JDK9引入的transferTo方法, 可以直接在两个流中传输数据  
                in.transferTo(out);  
            }  
        }  
    }  
}
```

此修改后的Server, 仍适用原来的Client端App。

使用消息头方式交换变长信息

```
Run: One2OneServer x One2OneClient x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" --module-path
正在监听端口9999,等待客户端连接.....
接收到客户端连接: Socket[addr=/127.0.0.1,port=12039,localport=9999]
数据输出完毕!
```

Server端等待客户端连接，一旦有客户端连接进来，就把一首古诗传给它。

```
Run: One2OneClient x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" --module-path
古诗一首
画
-----
远看山有色
近听水无声
春去花还在
人来鸟不惊
-----
数据接收结束
客户端退出

Process finished with
```

客户端接收到的古诗

示例代码位于one2one包中。

两个辅助工具函数

```
/**
 * int整数转换为4字节的byte数组
 */
1 usage
public static byte[] intToByte4(int i) {
    byte[] targets = new byte[4];
    targets[3] = (byte) (i & 0xFF);
    targets[2] = (byte) (i >> 8 & 0xFF);
    targets[1] = (byte) (i >> 16 & 0xFF);
    targets[0] = (byte) (i >> 24 & 0xFF);
    return targets;
}
```

```
/**
 * byte数组转换为int整数
 */
1 usage
public static int byte4ToInt(byte[] bytes, int off) {
    int b0 = bytes[off] & 0xFF;
    int b1 = bytes[off + 1] & 0xFF;
    int b2 = bytes[off + 2] & 0xFF;
    int b3 = bytes[off + 3] & 0xFF;
    return (b0 << 24) | (b1 << 16) | (b2 << 8) | b3;
}
```

消息编码

```
//按照UTF8编码消息，然后将消息长度放到消息开头（占4个字节），  
//返回处理好的字节数组  
public static byte[] getStringBytes(String message)  
    throws IllegalArgumentException {  
    if(message==null || message.length()==0) {  
        throw new IllegalArgumentException("要发送的消息不能为空");  
    }  
    try {  
        //按照UTF-8编码分解字符串为字节数组  
        byte[] messageData=message.getBytes("utf-8");  
        //将消息长度转为字节数组  
        byte[] messageLength=intToByte4(messageData.length);  
        //构建一个能容纳所有数据的数组  
        byte[] data=new byte[4+messageData.length];  
        //复制字符串长度数值  
        int index=0;  
        for(index=0;index<4;index++) {  
            data[index]=messageLength[index];  
        }  
        //复制消息数据  
        for(int i=0;i<messageData.length;i++) {  
            data[index+i]=messageData[i];  
        }  
        return data;  
    } catch (UnsupportedEncodingException e) {  
        throw new IllegalArgumentException("要发送的消息不能转换为UTF-8编码");  
    }  
}
```

消息解码

```
/**
 * 基于UTF-8编码从字节数组（是已经移除过前4个字节的原始数据）中还原数据
 */
public static String getStringFromUTF8ByteArray(byte[] data)
    throws UnsupportedOperationException {
    if (data == null || data.length == 0) {
        throw new IllegalArgumentException("要解码的数据能为空");
    }
    String message = new String(data, "utf-8");
    return message;
}
```

服务端核心代码

```
try (ServerSocket server = new ServerSocket(PORT)) {
    System.out.println("正在监听端口" + PORT + ",等待客户端连接.....");
    while (true) {
        try (Socket clientConnection = server.accept()) {
            System.out.println("接收到客户端连接: " + clientConnection);
            OutputStream outputStream = clientConnection.getOutputStream();
            List<String> messages = fillMessages(); //生成要发送的消息
            for (String message : messages) {
                //消息编码,检测每条消息的长度,追加到消息开头
                byte[] messageData = One2OneHelper.getStringBytes(message);
                outputStream.write(messageData); //发送消息
            }
            //输入4个字节,全0,表示输入结束
            byte[] end = {0, 0, 0, 0};
            outputStream.write(end);
            outputStream.flush();
            clientConnection.close();
            System.out.println("数据输出完毕! ");
        } catch (IOException e) {
        }
    }
} catch (Exception e) {
    System.out.println(e);
}
```

客户端核心代码

注意我们不能保证一定一次可以完整地接收4个字节拼接出消息长度，所以，应该使用循环的方式进行接收。

由于前面已经接收了4个字节，所以，后面就是真正的字符串数据了，可以直接解码。

```
try (Socket socket = new Socket(server, port)) {  
    // 反复接收消息，直到Server传回空消息（消息长度为0）  
    while (true) {  
        InputStream inputStream = socket.getInputStream();  
        byte[] messageLengthData = new byte[4];  
        int received = 0;  
        // 接收消息长度信息  
        while (received < 4) {  
            received += inputStream.read(messageLengthData, received, len: 4 - received);  
        }  
        int messageLength = One2OneHelper.byte4ToInt(messageLengthData, off: 0);  
        if (messageLength == 0) {  
            System.out.println("数据接收结束");  
            break;  
        }  
        // 接收消息数据  
        byte[] messageData = new byte[messageLength];  
        received = 0;  
        while (received < messageLength) {  
            received += inputStream.read(messageData, received, len: messageLength - received);  
        }  
        // 按照UTF-8格式解码收到的消息  
        String message = One2OneHelper.getStringFromUTF8ByteArray(messageData);  
        System.out.println(message);  
    }  
    System.out.println("客户端退出");  
} catch (IOException e) {...}
```

示例：基于Socket实现远程控制

Run: RemoteControlServer x	RemoteControlClient x
<pre>"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" --module-path "C 正在监听端口9999,等待客户端连接..... 接收到客户端连接: Socket[addr=/127.0.0.1,port=4747,localport=9999] 收到命令: now 当前时间已经传给客户端 收到命令: aaa 不能识别的命令 收到命令: quit Server退出 Process finished with exit code 0</pre>	<pre>"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" Server已连接 输入命令: now Server当前时间: Tue Jun 07 18:47:37 CST 2022 输入命令: aaa Server: 未知命令,无法执行 输入命令: quit Server:接收到退出请求,已退出! 客户端退出 Process finished with exit code 0</pre>

服务端响应客户端请求完成特定的工作

客户端通过命令窗口输入命令，
远程操控服务端应用程序

参看remotecontrol包中的示例

服务端核心代码

```
try (ServerSocket server = new ServerSocket(PORT)) {
    System.out.println("正在监听端口" + PORT + ",等待客户端连接.....");
    try (Socket clientConnection = server.accept()) {
        System.out.println("接收到客户端连接: " + clientConnection);
        var writer = new OutputStreamWriter(clientConnection.getOutputStream(), StandardCharsets.UTF_8);
        var out = new PrintWriter(writer, autoFlush: true);
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(
            clientConnection.getInputStream(), StandardCharsets.UTF_8));
        //接收客户端命令
        while (true) {
            String command = bufferedReader.readLine();
            System.out.println("收到命令: " + command);
            //解析命令
            if (command.equals(ControlCommands.QUIT)) {
                out.println("Server:接收到退出请求, 已退出! ");
                System.out.println("Server退出");
                break;
            } else if (command.equals(ControlCommands.NOW)) {
                out.println("Server当前时间: " + new Date());
                System.out.println("当前时间已经传给客户端");
            } else {
                out.println("Server: 未知命令, 无法执行");
                System.out.println("不能识别的命令");
            }
        }
    }
} catch (Exception e) {
}
} catch (Exception e) { ... }
```

//代表预先定义好的网络命令

3 usages

```
public class ControlCommands {
```

//获取当前时间命令

1 usage

```
public final static String NOW="now";
```

//退出命令

2 usages

```
public final static String QUIT="quit";
```

```
}
```

客户端核心代码

```
Scanner scanner = new Scanner(System.in);
try (Socket socket = new Socket(server, port)) {
    System.out.println("Server已连接");
    // 准备好读取和写入数据的流对象
    PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
    Reader reader = new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8);
    BufferedReader bufferedReader = new BufferedReader(reader);
    while (true) {
        System.out.print("输入命令: ");
        String userInput = scanner.nextLine();
        // 将命令发送到Server端
        if (userInput != null && userInput.length() > 0) {
            userInput = userInput.trim().toLowerCase();
            out.println(userInput);
            // 接收Server端发来的响应
            String response = bufferedReader.readLine();
            if (response != null) {
                System.out.println(response);
                // 如果是退出命令并且Server端已经成功发来响应, 则退出
                if (userInput.equals(ControlCommands.QUIT)) {
                    break;
                }
            }
        }
    }
    System.out.println("客户端退出");
    scanner.close();
}
```

一问一答工作方式

我们可以进一步完善本示例程序，允许客户端一次发送多条命令给服务端（比如，可以使用“;”分隔开多条命令）。

服务端收到客户端发来的消息，基于“;”切割出单条命令，执行完毕之后，如果有多个结果，则可以采用“消息头”的方式，将多个结果分为多个消息，发给客户端。

客户端收到服务端发来的消息后，基于“消息头”知道每条结果的长度，就能成功地从接收到信息中，正确地切割出每个结果。